

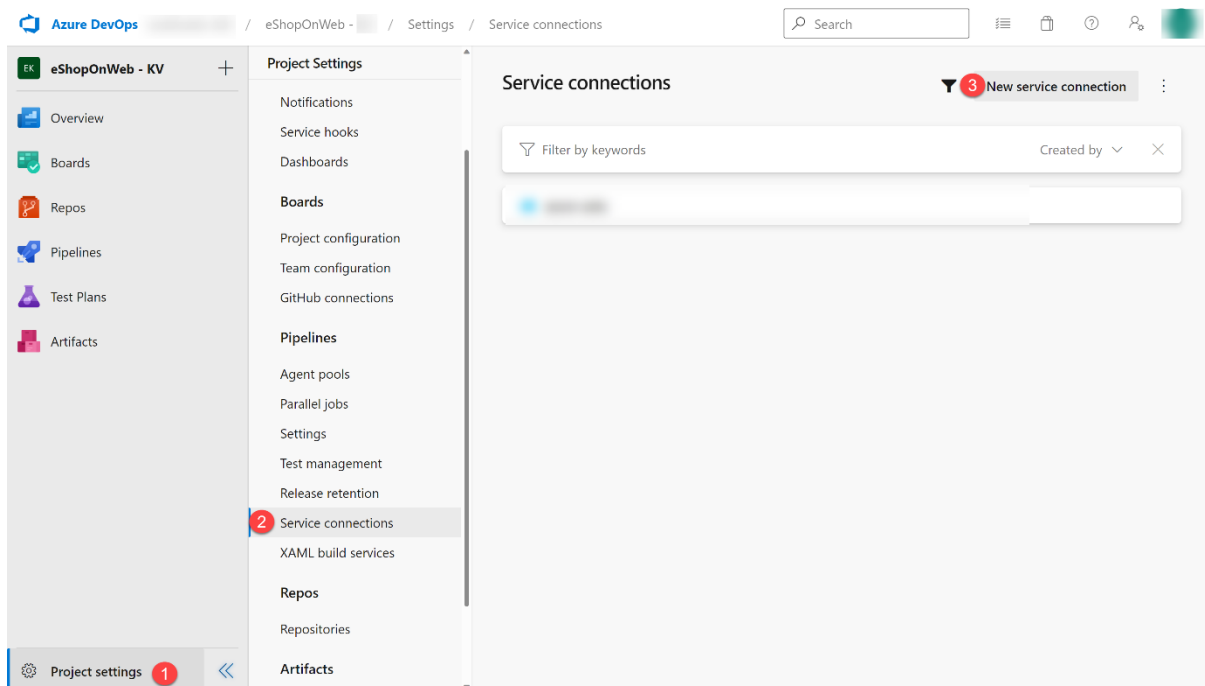
Create a service connection to access Azure resources

You will need to create a service connection in Azure DevOps which will allow you to deploy and access resources in your Azure subscription.

1. Start a web browser, navigate to the Azure DevOps portal <https://aex.dev.azure.com>.
2. Sign in to the Azure DevOps organization with the **Username** and **TAP** present on the **Resources** tab.

Note: If this is the first time you are signing in to the Azure DevOps organization, you will be prompted create your profile and accept the terms of service, and then select **Continue**.

3. Open **eShopOnWeb** project, and select **Project settings** in the bottom left corner of the portal.
4. Select the **Service connections** under Pipelines, and then select **Create service connection** button.



5. On the **New service connection** blade, select **Azure Resource Manager** and **Next** (you may need to scroll down).
6. Select **App registration (automatic)** from the **Identity type** dropbox.
7. Select **Workload Identity federation** and **Subscription** under the **Scope level**.

Note: You can also use **App registration or managed identity (manual)** if you prefer to manually configure the service connection. Follow the steps in the [Azure DevOps documentation](#) to create the service connection manually.

8. Fill in the empty fields using the information:

- **Subscription:** Select your Azure subscription.
- **Resource group:** rg-eshoponweb
- **Service connection name:** Type **azure subs**. This name will be referenced in YAML pipelines to access your Azure subscription.

9. Make sure the **Grant access permission to all pipelines** option is unchecked and select **Save**.

Important: The **Grant access permission to all pipelines** option is not recommended for production environments. It is only used in this lab to simplify the configuration of the pipeline.

Note: If you see an error message indicating you don't have the necessary permissions to create a service connection, try again, or configure the service connection manually.

Implement GitHub Actions for CI/CD

In this lab, you'll learn how to implement a GitHub Action workflow that deploys an Azure web app. You'll work with GitHub repositories, Azure Service Principals, and automated deployment workflows.

You will learn how to:

- Import and configure a GitHub repository for CI/CD.
- Create an Azure Service Principal for GitHub Actions authentication.
- Implement a GitHub Action workflow for CI/CD.
- Set up GitHub environments with manual approval gates.
- Deploy applications to Azure using GitHub Actions.

Before you start

To complete the lab, you need:

- **Microsoft Edge** or an [Azure DevOps supported browser](#).
- **An Azure subscription:** If you don't already have an Azure subscription, sign up for a free account at [Azure Free Account](#).
- Verify that you have a Microsoft account or a Microsoft Entra account with the **Contributor** or the **Owner** role in the Azure subscription. For details, refer to [List Azure role assignments using the Azure portal](#) and [View and assign administrator roles in Azure Active Directory](#).
- **A GitHub account:** If you don't already have a GitHub account that you can use for this lab, follow instructions available at [Signing up for a new GitHub account](#) to create one.

Import eShopOnWeb to your GitHub Repository

In this section, you will import the existing [eShopOnWeb](#) repository code to your own GitHub public repository.

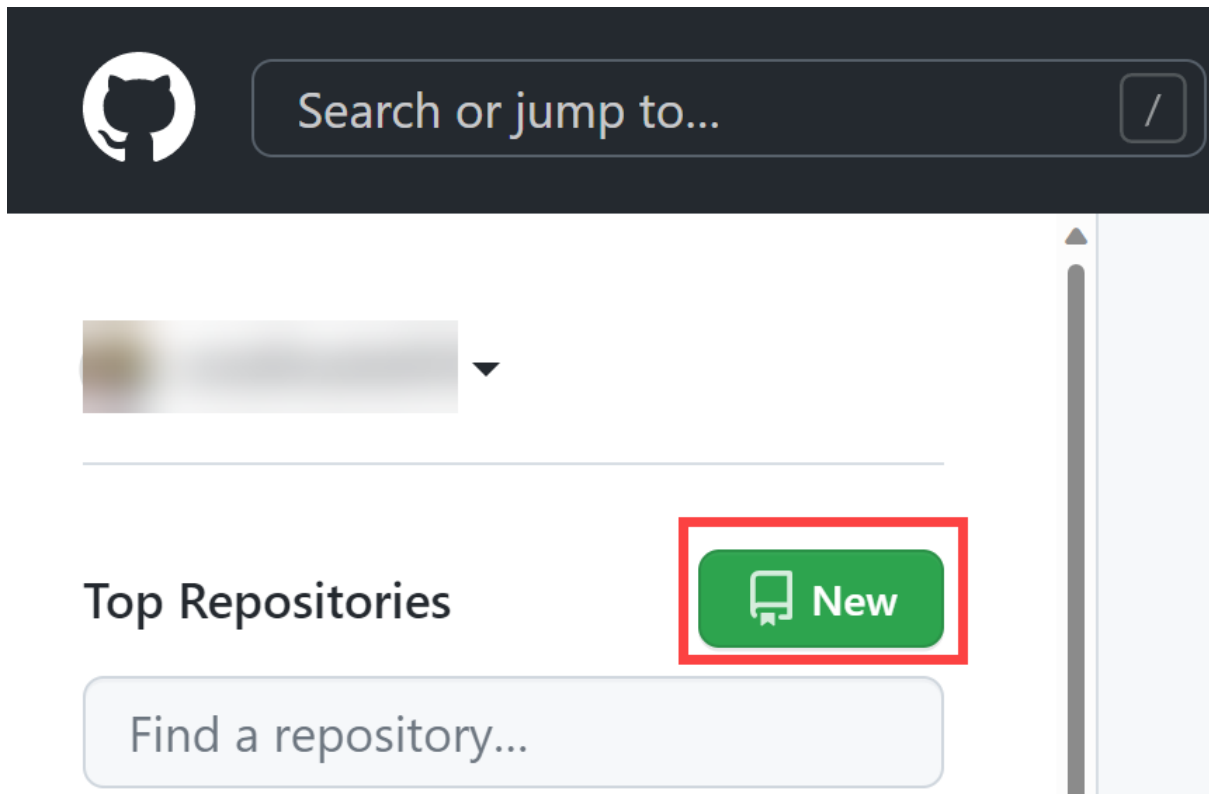
The repository is organized the following way:

- **.ado** folder contains Azure DevOps YAML pipelines.
- **.devcontainer** folder container setup to develop using containers (either locally in VS Code or GitHub Codespaces).
- **infra** folder contains Bicep & ARM infrastructure as code templates used in some lab scenarios.
- **.github** folder container YAML GitHub workflow definitions.
- **src** folder contains the .NET 8 website used on the lab scenarios.

Create a public repository in GitHub and import eShopOnWeb

In this task, you will create an empty public GitHub repository and import the existing [eShopOnWeb](#) repository.

1. From the lab computer, start a web browser, navigate to the **GitHub website** <https://github.com/>, sign in using your account and select on **New** to create new repository.



2. On the **Create a new repository** page, select on **Import a repository** link (below the page title).

Note: you can also open the import website directly at <https://github.com/new/import>

3. On the **Import your project to GitHub** page:

Field	Value
The URL for your source repository	<code>https://github.com/MicrosoftLearning/eShopOnWeb</code>
Owner	Your account alias
Repository Name	eShopOnWeb
Privacy	Public

4. Select on **Begin Import** and wait for your repository to be ready.
5. On the repository page, go to **Settings**, select on **Actions > General** and choose the option **Allow all actions and reusable workflows**. Select on **Save**.

- General
- Access
- Collaborators
- Code and automation
- Branches
- Tags
- Actions
- General 2
- Runners

Actions permissions

3

☒ Allow all actions and reusable workflows

Any action or reusable workflow can be used, regardless of who authored it or where it is defined.

☐ Disable actions

The Actions tab is hidden and no workflows can run.

☐ Allow actions and reusable workflows

Any action or reusable workflow defined in a repository within unaihuete93 can be used.

☐ Allow and select , actions and reusable workflows

Any action or reusable workflow that matches the specified criteria, plus those defined in a repository within unaihuete93, can be used. [Learn more about allowing specific actions and reusable workflows to run.](#)

Save 4